UNIVERSITÀ DI BOLOGNA

School of Engineering

Master Degree in Automation Engineering

Image Processing and Computer Vision

Fruit Inspection

Professor: **Luigi Di Stefano**

Students:
Fardeen Kadri
Naveen Joseph Delphin

Academic year 2024/2025

Abstract

The following project, proposed by UNITEC S.p.A., focuses on the automatic inspection of fruits using computer vision techniques. The aim is to identify surface defects such as bruises, russets, or imperfections through the analysis of both near-infrared (NIR) and RGB images captured from a conveyor belt setup. The data set includes multiple pairs of images with minimal parallax between the NIR and color views. The project is divided into three tasks: fruit segmentation and general defect detection, russet detection, and kiwi inspection. Each task emphasizes different technical challenges, including segmentation, color space transformation, statistical modeling, and morphological analysis. The overall objective is to develop robust image processing pipelines capable of detecting anomalies while minimizing false positives, thereby aiding automated quality control in agricultural sorting lines.

Contents

Introduction

In modern agriculture and food industries, automated inspection systems are becoming increasingly important for ensuring the quality and uniformity of products. Among these, fruit inspection stands out as a critical application area where defects such as bruises, russets, and surface blemishes directly impact product grading, market value, and consumer acceptance. Traditional manual inspection methods are often slow, inconsistent, and labor-intensive, prompting the need for efficient computer vision-based solutions.

This project, proposed by UNITEC S.p.A., addresses the challenge of automated fruit quality inspection by leveraging image processing and machine learning techniques. Each fruit sample is captured using both Near Infra-Red (NIR) and RGB cameras, providing complementary information for segmentation and defect analysis, as shown below:

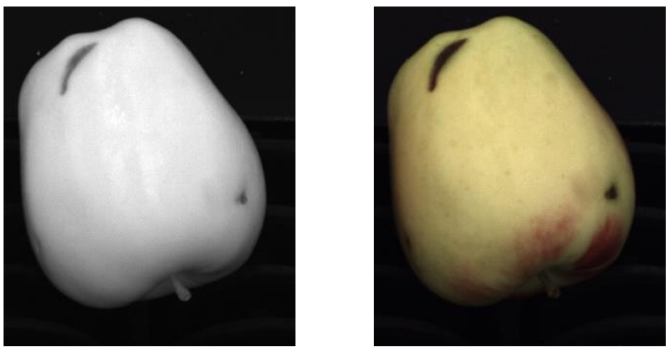


Figure 1: NIR and color picture

The project is structured into three tasks: general defect detection, russet detection, and kiwi inspection. Each task presents unique technical challenges related to color segmentation, statistical modeling, and robustness against variable image conditions.

1. FIRST TASK: FRUIT SEGMENTATION AND DEFECT DETECTION

1.1 Introduction

First task involves outlining the fruit through the creation of a binary mask. For performing this task, it is hinted to first apply a threshold to the entire image to eliminate the background while preserving the fruit's borders and use the flood-fill technique to fill any holes inside the fruit region, ensuring a clean and solid mask of the fruit. Since the image pairs exhibit minimal parallax, I was able to compute the mask on one image and apply it to the other.

Next, we have to search for defects on each fruit. It is recommended to focus on identifying areas with strong edge features, which are typical indicators of the defects.

1.2 Fruit Segmentation Using Otsu Thresholding

1.2.1 Gaussian Blurring for Preprocessing:

In this part, fruit segmentation is performed on Near Infrared (NIR) images to isolate the fruit from its background. The segmentation step is important for doing accurate defect detection, as it ensures us that only the pixels of fruit are processed making it optimal for defect detection. The segmentation process begins by applying Gaussian Blur to the grayscale NIR image to reduce noise and smoothen the image so that the thresholding is accurate as told by the professor in the class.

1.2.2 Binary Thresholding:

For thresholding we used the Otsu's thresholding method. Otsu's method works by analyzing the histogram of pixel intensities and selecting a threshold that minimizes intra-class variance, effectively separating fruit from

background in cases where the image histogram exhibits a bimodal distribution-meaning the foreground and background intensities form two distinct peaks. The binary image that results from this process often contains small holes or gaps within the fruit region due to surface texture or inconsistent lightning.

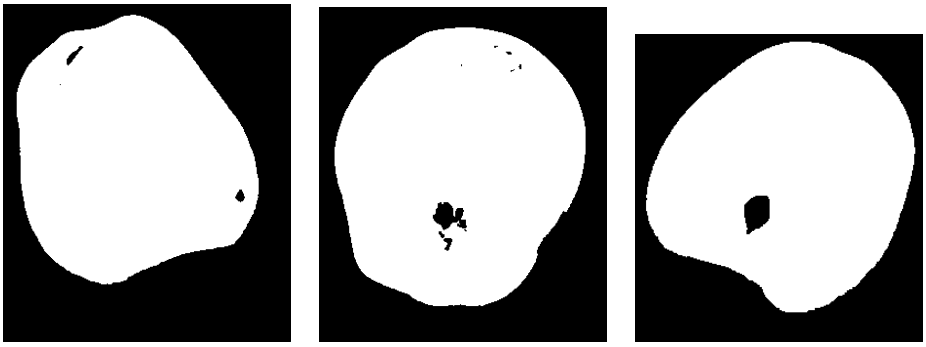


Figure 2: Otsu Thresholding

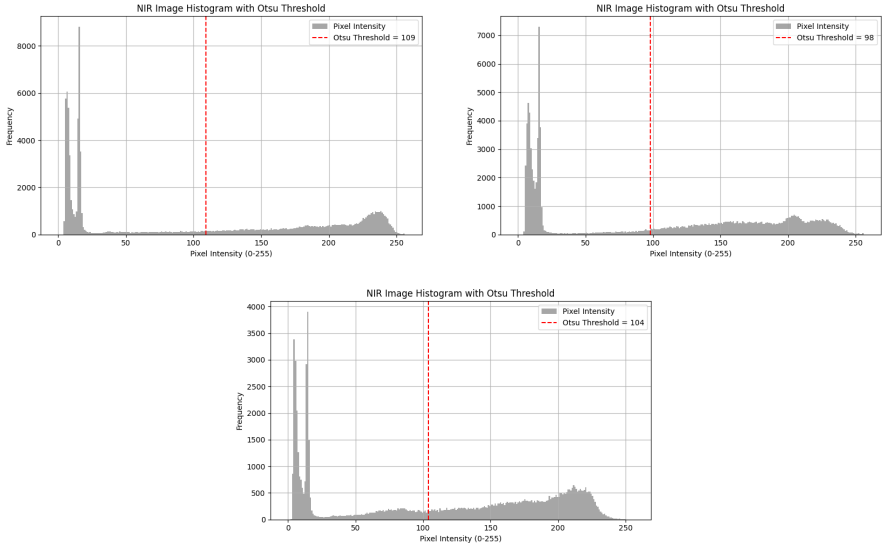


Figure 3: Otsu Threshold Histograms with values

1.2.3 Flood-Filling:

The binary image that results from the thresholding process contains small holes or gaps within the fruit region due to surface texture or lighting inconsistencies.

To address these artifacts, a flood-fill operation is employed starting from the top-left corner of the image. Before flood-filling, a temporary mask is

created that is slightly larger than the original image as OpenCV requires the flood fill mask to be 2 pixels larger in both dimensions than the source image. The flood-fill fills all background-connected black areas with white, effectively labeling the background.

So the inverting operation was implemented on the flood-filled image resulting in the previously unfilled regions inside the fruit blob, essentially the hole, highlighted to white. These are then combined with the original binary mask using a bitwise OR operation, resulting in a complete and hole free binary mask of the fruit. This final mask is much more robust for further processing.

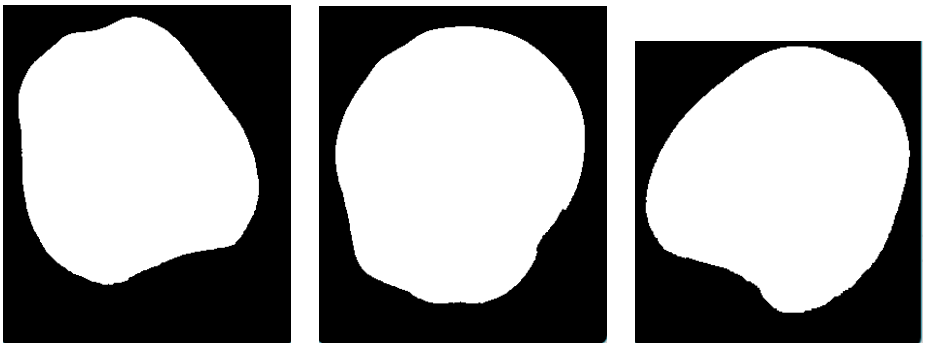


Figure 4: Final Binary Mask

1.3 Defect Detection and Final Representation

1.3.1 Edge Detection

The first step involves detecting the edges of defects on the fruit's surface. Firstly, the bilateral filter is applied to the segmented NIR image. This filter is effective in smoothing the image while preserving edges, which is crucial for accurate edge detection. The filtered image is then passed to the Canny edge detection algorithm with thresholds selected empirically (50 and 120). Canny edge detection is chosen here for its precision in identifying sharp intensity changes, which are often indicative of surface defect.

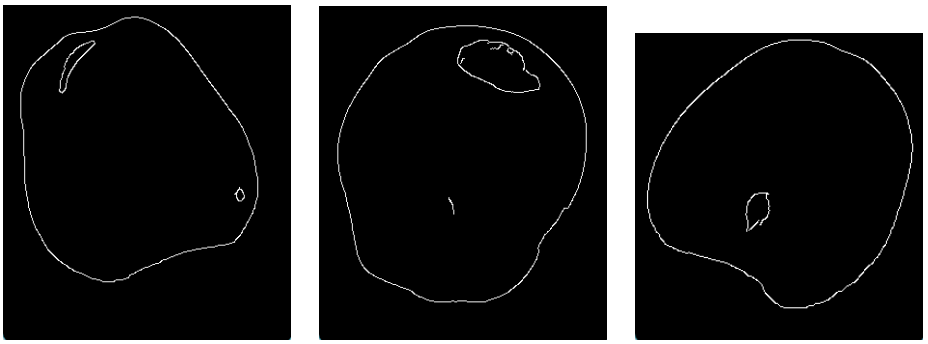


Figure 5: Canny's Edge Detection

Minkowski subtraction was also explored for edge detection. However, after practical implementation and testing, this approach introduced excessive noise, especially in the smooth regions of the fruit surface, which led to false positives and inconsistent defect outlines. However the results of minkowski subtraction can be refined better by sharpening and high thresholding the image which removes the false edges and noises. In contrast, the Canny edge detection method, especially when combined with bilateral filtering, provided much cleaner edge maps, capturing the darker defect regions with greater accuracy. Due to these performance differences, the Minkowski subtraction-based method was ultimately not included in the final defect detection workflow, although it produced a reasonable output.

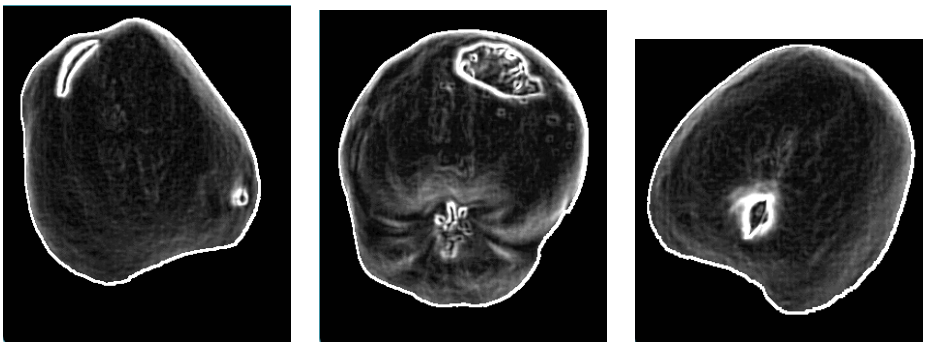


Figure 6: Minkowski Subtraction

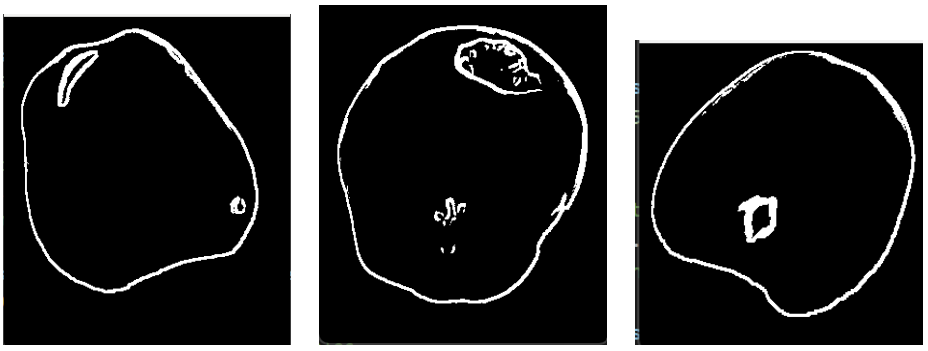


Figure 7: Minkowski Edge Detection After Refining

After identifying the edges, morphological closing operation was applied using a 5x5 cross-shaped structuring element. The closing operation consists of a dilation followed by an erosion. This helps in connecting broken edges if any and ultimately filling small gaps within detected contours resulting in a more cohesive binary image.

1.3.2 Extraction and Isolation of Defect Regions

To separate the defective regions from the non-defective parts of the fruit, a flood-fill technique is used. First, the binary image is converted to a proper type for flood-filling. A new mask is created by inverting the original fruit mask and padding it with rows and columns of zeros. This padding ensures that the flood-fill algorithm has enough space around the edges of the image to work correctly. The flood-fill operation starts from a point (175, 150) which lies within the background or non-defective area (found after manual testing of different values), filling all connected areas with white (255). The output of this operation highlights the non-defective regions of the image.

After identifying the non-defective regions, the image is inverted so that the defect areas become white, and everything else becomes black. This image is then combined with the original fruit mask using logical AND operation to ensure only regions within the fruit are considered. To refine the result further and remove small noise or irrelevant blobs, a morphological opening operation is applied using a 3x3 rectangular structuring element. The final output is a clean binary image showing only the isolated defects on the fruit surface.

Figure 8: Binary Image with Isolated Defects Alone

1.3.3 Final Representation of Defects

In the final steps, the contours in the binary image, where defects are highlighted, are identified. Only the external contours are retrieved to focus solely on the main defect boundaries. To improve visualization, each defect region is enclosed within a minimum enclosing circle, which provides a clean, rounded outline around irregular defect shapes. These circles are drawn on a blank image using a green color. The generated contour image is then superimposed on the original color image using weighted addition, that highlights the defect regions.

Figure 9: Final Outcome By Canny Edge Detection

Figure 10: Final Outcome By Minkowski Subtraction

2. SECOND TASK: RUSSET DETECTION

2.1 Introduction:

Detecting russet defects on apples poses a challenge due to their subtle coloration and irregular distribution. The defects manifest as light brown, rough patches that can be mistaken for natural apple texture or shadows. In this task, we aim to implement a robust pipeline for isolating russet regions from RGB and NIR images of apples using a combination of image preprocessing, clustering, and statistical analysis.

A particular focus is placed on comparing color spaces to determine the most effective representation for color-based segmentation. Our method applies segmentation through thresholding on NIR images, K-means clustering in various color spaces, and refinement with Mahalanobis distance in the most advantageous color space.

2.2 Dataset and Input Structure

The dataset consists of image pairs:

- **NIR Images**: Grayscale representations enhancing textural and surface-level contrast.
- **Color Images**: Captured in RGB (BGR in OpenCV) format, containing visible light details.

Each image pair is named following the pattern:

- C0_00000x.png: NIR image
- C1_00000x.png: RGB image

Where x is the image index (e.g., 4 or 5).

2.3 Fruit Segmentation using NIR

The first crucial step in russet detection is to isolate the fruit from the background in the image so that further processing (like clustering or distance measurement) happens only on the relevant regionâthe apple itself. This is done using the Near Infrared (NIR) image because:

NIR enhances contrast between the fruit and the background.Surface texture and brightness patterns are more distinct than in RGB.

The segmentation process includes the following steps:

2.3.1 Gaussian Blurring

```
blur_nir = cv2.GaussianBlur(nir_image, (3, 3), 0)
```

Listing 1: Gaussian Blurring

Purpose: To reduce high-frequency noise in the NIR image before thresholding. A small 3x3 kernel is used to preserve the edges while smoothing the slight irregularities at the pixel level.

Why it's needed: Noise or harsh gradients in the raw NIR image can cause erratic thresholding. Blurring creates more coherent regions for binarization.

2.3.2 Thresholding

```
th, thresh = cv2.threshold(blur_nir, 50, 255, cv2.THRESH_BINARY
    )
```

Listing 2: Thresholding

Purpose: To convert the grayscale image into a binary mask where:

- White (255) = fruit
- Black (0) = background

The threshold value of 50 is manually set based on visual inspection of the NIR histogram. Optionally, Otsu's thresholding can be used to automatically select this value:

```
otsu_value = cv2.threshold(nir_image, 0, 255, cv2.THRESH_BINARY
    + cv2.THRESH_OTSU)[0]
```

Listing 3: OTsu's Thresholding

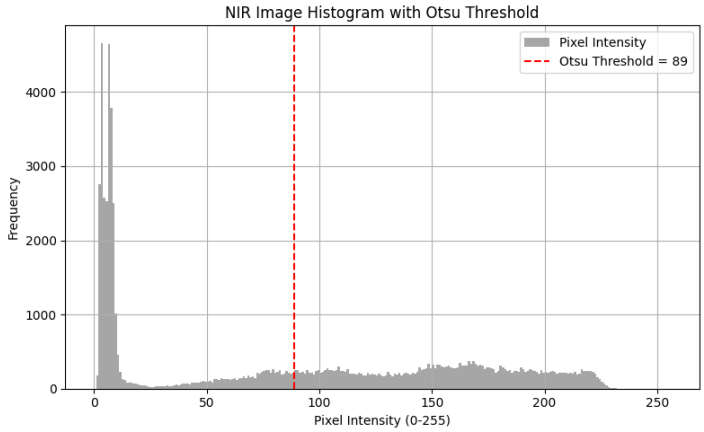


Figure 11: Histogram for picture 4

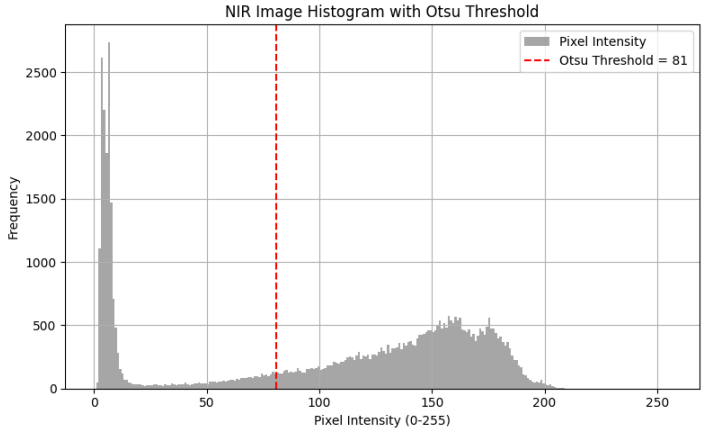


Figure 12: Histogram for picture 5

Why not Otsu in the final code? Because the apples may vary in lighting and contrast across samples, a fixed value of 50 was empirically found to generalize better in your test images.

2.3.3 Hole Filling using Flood-Fill

```
m1 = np.zeros((h+2, w+2), np.uint8)
ff1 = thresh.copy()
cv2.floodFill(ff1, m1, (0, 0), 255)
```

Listing 4: OTsu's Thresholding

Purpose: To detect the background (starting in the upper left corner (0,0)) and fill it white. This step identifies what is not part of the fruit, assuming that the fruit does not touch the edges of the image.

Then we invert the flood-filled image:

```
holes = cv2.bitwise_not(ff1)
```

Listing 5: OTsu's Thresholding

Now, holes contains the internal voids inside the fruit mask (e.g., stem holes or minor dark regions inside the apple), marked in white.

2.3.4 Final Mask Creation

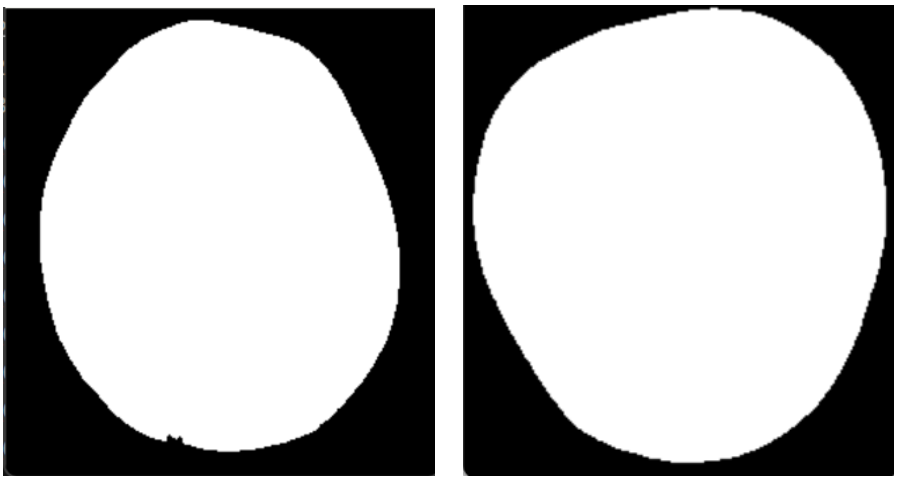
```
mask_nir = holes | thresh
```

Listing 6: OTsu's Thresholding

This bitwise OR operation merges:

- The thresholded image (thresh) that outlines the fruit blob
- The filled holes (holes) that patch gaps within the fruit

Result: a **clean binary mask** (mask_nir) that represents the whole apple without internal black spots or holes.



(a) Mask of 4 (b) Mask of 5

Figure 13: Mask generated

2.3.5 Applying the Mask to RGB Image

```
mask_rgb = cv2.cvtColor(mask_nir, cv2.COLOR_GRAY2RGB)
bool_mask_nir = ones & mask_rgb
app_rgb = rgb_image * bool_mask_nir.astype(np.uint8)
```

Listing 7: OTsu's Thresholding

- The grayscale mask is extended to three channels to match the RGB image.
- A boolean multiplication is applied to remove the background and retain only the fruit pixels.

Why it matters: This step ensures that only fruit regions are passed to the K-means clustering stageâavoiding distractions from the conveyor belt, lighting artifacts, or other objects.

2.4 Comparative Study of Color Spaces:

In order to identify the optimal color space for russet segmentation, the `color_space_comparison()` function was implemented. It performs:

- **Color Histogram Analysis**: Intensity distributions for each channel in RGB, HSV, LAB, LUV, and HLS color spaces.
- **3D Color Scatter Plots**: Pixels are plotted in 3D space using their respective color channels (e.g., A-B-L in LAB).

2.4.1 Observations from Color Space Comparison

The comparative study was conducted on both sample images (`C1_000004` and `C1_000005`):

- **RGB (BGR in OpenCV)**:
  - No identifiable clusters or separable patterns across any 2D or 3D combinations.
  - Not useful for russet segmentation.

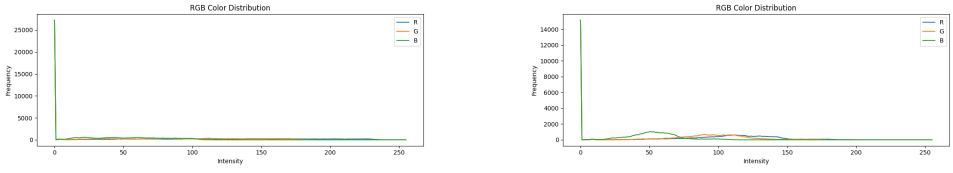


Figure 14: RGB Color distribution of images 4 & 5

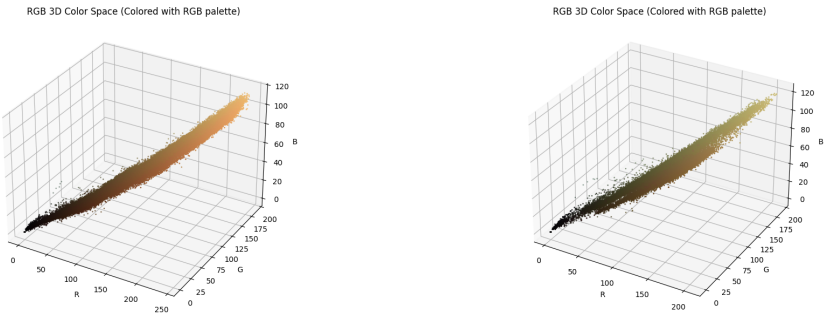


Figure 15: RGB 3D Color Space of images 4 & 5

- **HSV**:
  - Slight clustering observed in the **Hue-Saturation** channels for `C1_000005`.
  - No clear patterns in `C1_000004`
  - Unreliable due to brightness sensitivity.

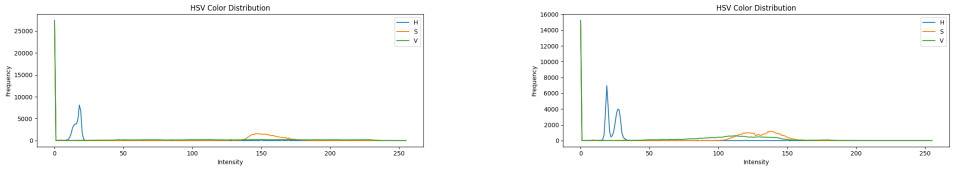


Figure 16: HSV Color distribution of images 4 & 5

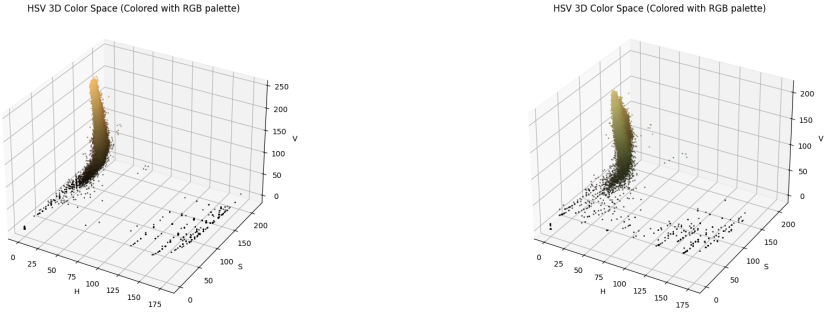


Figure 17: HSV 3D Color Space of images 4 & 5

- **HLS**:
  - Similar behavior to HSV.
  - H and S channels show better separation than L.
  - Not consistent across both apples.

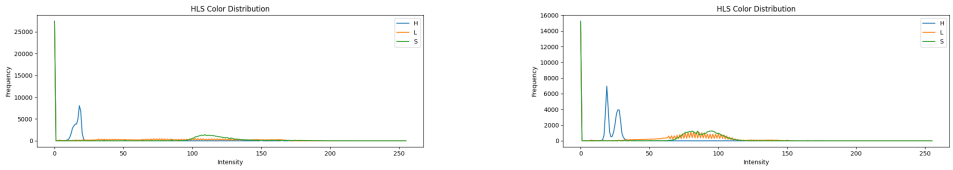


Figure 18: HLS Color distribution of images 4 & 5

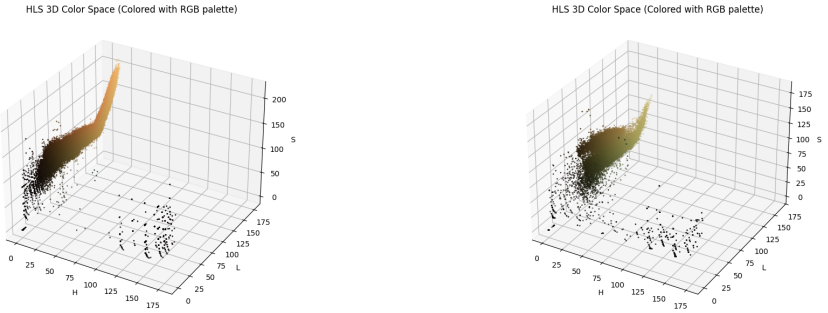


Figure 19: HLS 3D Color Space of images 4 & 5

- **LUV**:
  - **U and V channels** show denser regions of color in `C1_000005`.
  - Promising candidate for further exploration.

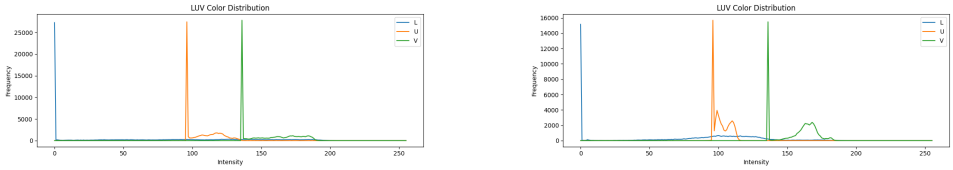


Figure 20: LUV Color distribution of images 4 & 5

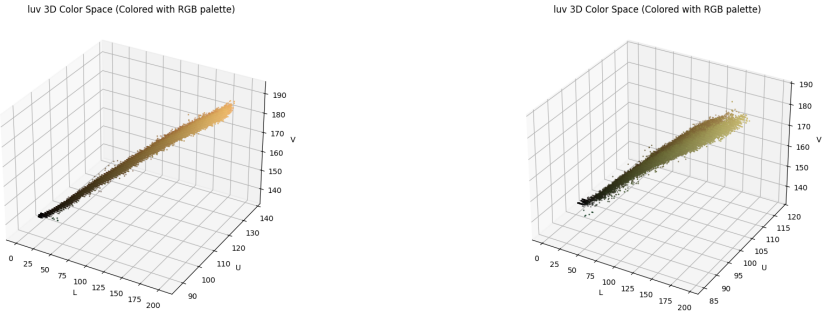


Figure 21: LUV 3D Color Space of images 4 & 5

- **LAB**:
  - Clear distinction of clusters in **A and B channels** for both images.

– Especially strong separation in `C1_000004`, even more than LUV.
– 2D histograms reveal well-separated dense regions corresponding to russet and healthy areas.

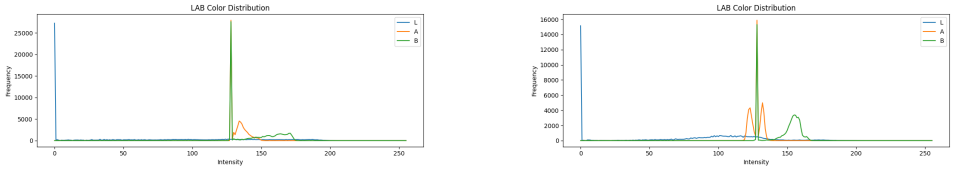


Figure 22: LAB Color distribution of images 4 & 5

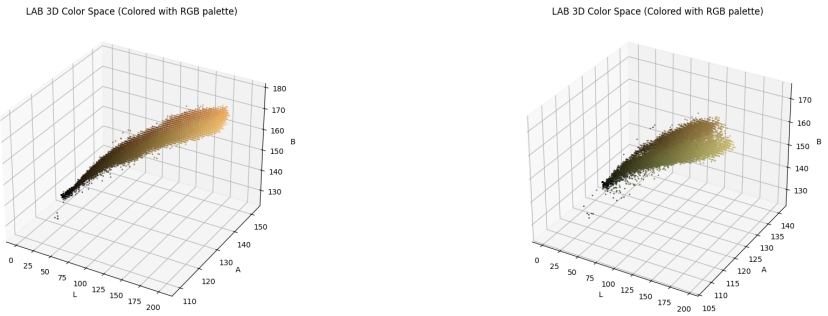


Figure 23: LAB 3D Color Space of images 4 & 5

2.4.2 Conclusion on Color Space Selection

Based on histogram plots and 3D visualization:

- **LAB** is the most effective color space for russet detection, offering the most distinguishable clustering between russet and healthy regions.
- **LUV** also provides promising results but is slightly less consistent.
- **HSV** and **HLS** are unreliable and should be avoided due to their poor cluster separation and brightness sensitivity.
- **RGB** lacks any meaningful separation and is unfit for defect segmentation.

Thus, the final detection pipeline uses the LAB space for K-means and Mahalanobis distance refinement.

2.5 Color-Based Clustering:

Once the fruit has been segmented from the background, the color information is used to identify potential russet regions. To achieve this, the masked RGB image is first converted into the LAB color space. Unlike the traditional RGB format, LAB separates luminance (lightness) from chromatic

components, which makes it particularly suitable for color clustering and segmentation.

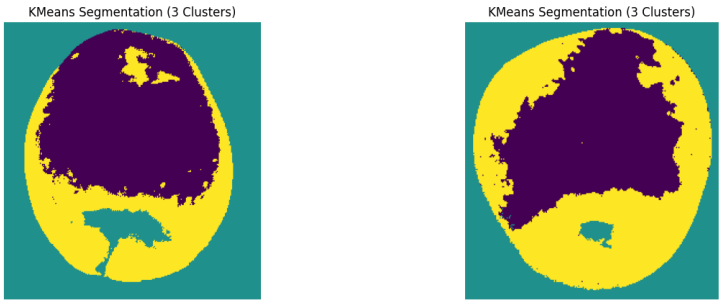
To isolate color differences, only the A (green-red) and B (blue-yellow) channels are considered for further analysis, while the L (luminance) channel is temporarily excluded. These two channels are reshaped into a two-dimensional array where each row corresponds to the A and B values of a pixel. This data structure is used as input to the **K-means clustering algorithm**.

2.5.1 Theory of K-Means Clustering

K-means is a popular unsupervised learning algorithm used to partition data into k clusters based on feature similarity. It operates by randomly initializing k centroids in the feature space, then iteratively assigning each data point to the nearest centroid based on Euclidean distance. After all points are assigned, the centroids are recomputed as the mean of all points in each cluster. This process repeats until convergence, i.e., when the assignments no longer change or when a maximum number of iterations is reached.

In this context, each pixel in the AB space is treated as a data point, and the algorithm attempts to group pixels into $k = 3$ distinct color clusters. The clustering allows us to identify different surface regions of the apple based on chromatic differences, which may include healthy skin, shadows, and potential russet areas.

After clustering, the original spatial arrangement of the pixels is restored to produce a segmented image where each region corresponds to a color cluster. To determine which cluster represents russet, the luminance (L) values of each cluster are analyzed. The assumption is that russet regions, which are slightly raised and rougher, tend to reflect more light and thus have higher L values. Therefore, the cluster with the highest average luminance is selected as the russet candidate.



(a) Image 4 (b) Image 5

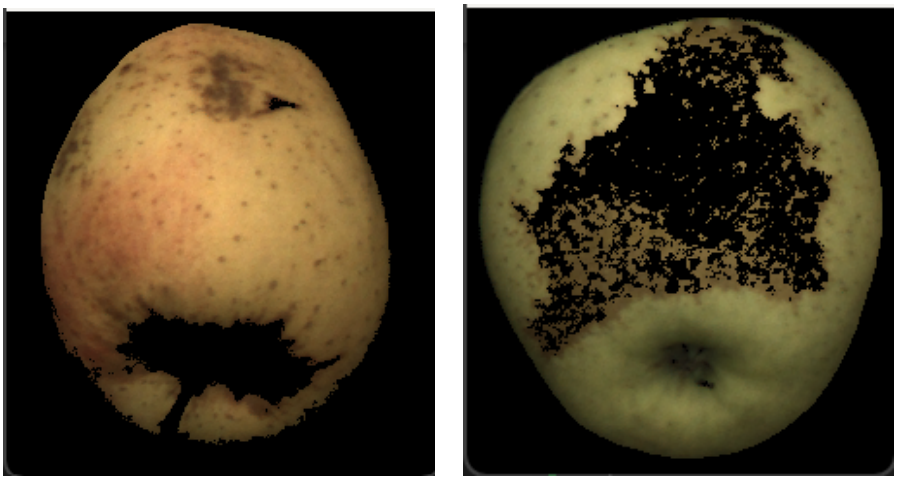
Figure 24: kmeans Segmentation

2.6 Refinement using Mahalanobis Distance

While K-means provides a good initial segmentation, it is not always precise in capturing the boundaries of the russet regions. There may be noise or misclassified pixels due to overlapping color characteristics between healthy skin and russet areas. To address this, the algorithm employs Mahalanobis distance as a statistical method for refining the segmentation.

Mahalanobis distance is a multivariate measure that considers the correlation and variance within the data. It measures how far a point is from the center of a distribution, taking into account the spread and orientation of the data cloud. In this task, the A and B values of the russet cluster are used to compute the mean vector and covariance matrix, which define the statistical "shape" of the russet region in AB space.

For each pixel in the fruit area, the Mahalanobis distance to the russet distribution is calculated. Pixels with a distance below a chosen threshold (empirically set to 5) are considered part of the russet region. This step significantly improves the segmentation, eliminating false positives and capturing the core russet area with higher precision. The resulting binary mask is applied to the original RGB image to visualize the final russet detection.



(a) Image 4 (b) Image 5

Figure 25: Russet Detected

3. FINAL CHALLENGE: KIWI INSPECTION

3.1 Introduction

The final task of the fruit inspection project is focused on the detection of surface-level defects on kiwi fruits. Compared to other fruits such as apples, kiwis present unique visual challenges. Their dark, textured skin with varying natural patterns makes it harder to visually differentiate between healthy regions and actual defects. Additionally, due to the hairy and rugged exterior of the kiwi, simple thresholding or color-based techniques are often insufficient.

This task utilizes a computer vision pipeline combining thresholding, morphological filtering, Canny edge detection, and contour analysis. The primary objective is to accurately segment the kiwi fruit from the background using Near-Infrared (NIR) imaging and subsequently identify irregular surface patches that might indicate bruises, abrasions, or other defects. The use of classical vision methods such as Otsu thresholding and Canny edge detection ensures a balance between performance and interpretability, making the pipeline suitable for real-time inspection systems.

3.2 Dataset and Input Description

Each kiwi sample is represented by two images:

- A **Near-Infrared (NIR) image**, stored as a grayscale file with prefix `C0_00000x.png`, captures surface reflectance and is particularly useful for distinguishing the fruit body from the background.
- A **Color (RGB) image**, stored as `C1_00000x.png`, provides the visible-light view of the fruit, used mainly for final visualization and defect marking.

The dataset contains sample images numbered from 6 to 10. The NIR and RGB images are taken from approximately the same viewpoint with

minimal parallax, allowing pixel-wise correlation between them. This alignment is leveraged to extract fruit contours in NIR and apply the same masks to the RGB images.

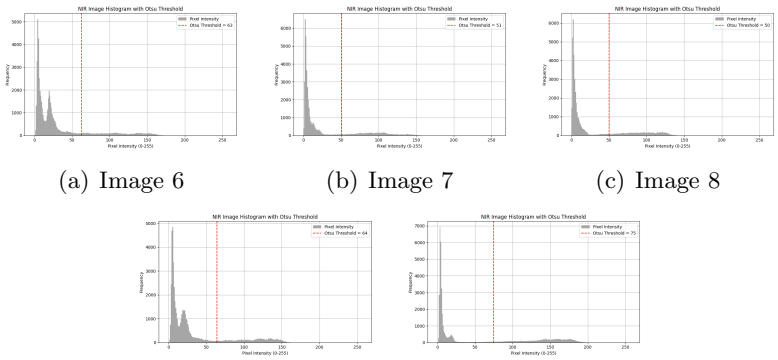
3.3 Fruit Segmentation Using Otsu Thresholding

3.3.1 Gaussian Blurring for Preprocessing

Segmentation starts with preprocessing the NIR image to remove high-frequency noise. A Gaussian blur is applied, which smoothens the image while preserving the edges. The blurring process is essential to reduce false edges and improve the robustness of the subsequent thresholding step.

3.3.2 Otsu Thresholding Theory and Application

To segment kiwi from the background, we apply Otsu method for automatic threshold selection. The Otsu method is a histogram-based algorithm that chooses a threshold that minimizes the variance between classes while maximizing the variance between classes. This approach assumes that the image histogram is bimodal, which is often valid for NIR images where the background and fruit appear as two distinct intensity regions.

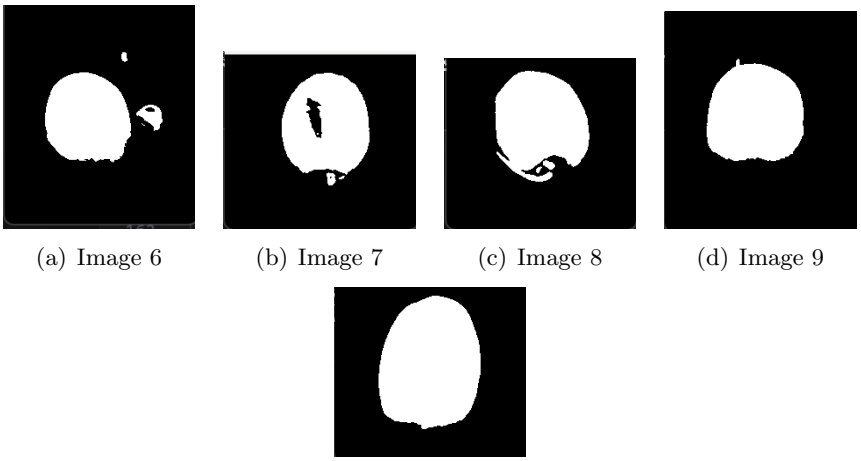


(a) Image 6 (b) Image 7 (c) Image 8

(d) Image 9 (e) Image 10

Figure 26: Histogram

The resulting binary mask separates the fruit blob (foreground) from the background (mostly dark in NIR). This mask is further cleaned using flood fill, which fills internal holes, and bitwise operations to unify the mask.



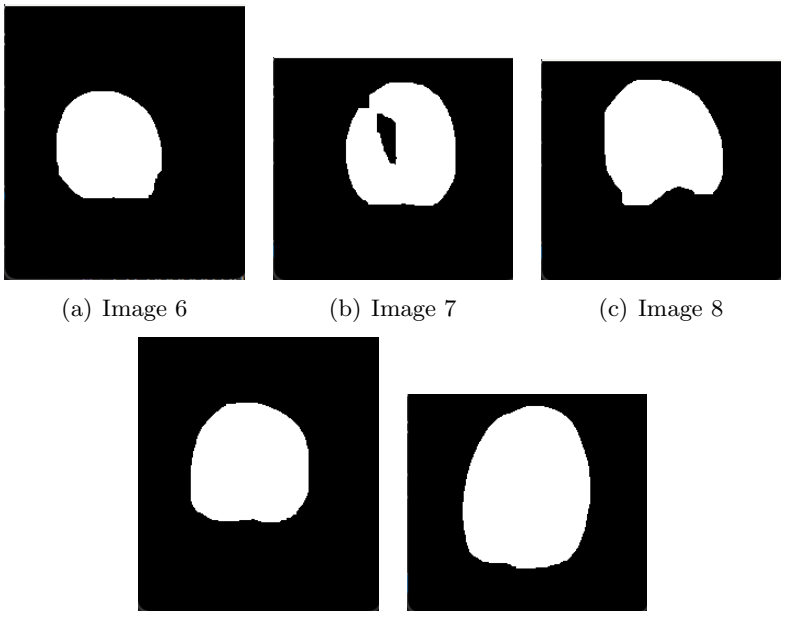
(a) Image 6 (b) Image 7 (c) Image 8 (d) Image 9

(e) Image 10

Figure 27: Flood Filled masks

3.3.3 Morphological Operations for Mask Refinement

To remove small noise or spurious blobs and ensure that only the fruit body remains, a morphological opening operation is performed. Using a large rectangular kernel, this process eliminates small connected components and smooths the outer contour of the kiwi, resulting in a clear fruit mask.

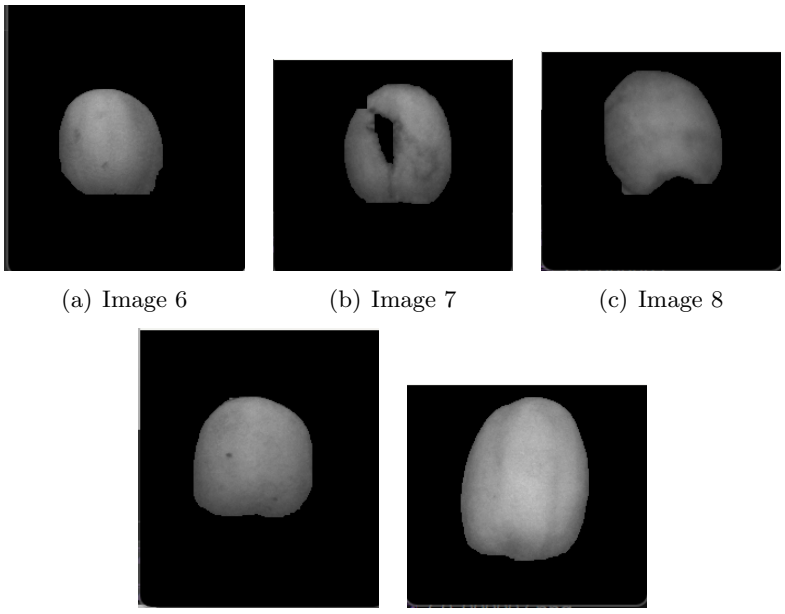


(a) Image 6 (b) Image 7 (c) Image 8

(d) Image 9 (e) Image 10

Figure 28: Mask after Opening

The final fruit mask is used both for limiting the region of interest (ROI) in defect detection and for excluding background pixels in further processing.



(a) Image 6 (b) Image 7 (c) Image 8

(d) Image 9 (e) Image 10

Figure 29: Mask applied to NIR images

3.4 Defect Detection Using Canny Edge Detection

3.4.1 Edge Detection in Computer Vision

Edges are locations of strong intensity change, often corresponding to boundaries of objects or regions with different textures. In fruit inspection, surface defects often manifest as regions with sharp intensity gradients, making edge detection a natural choice for defect localization.

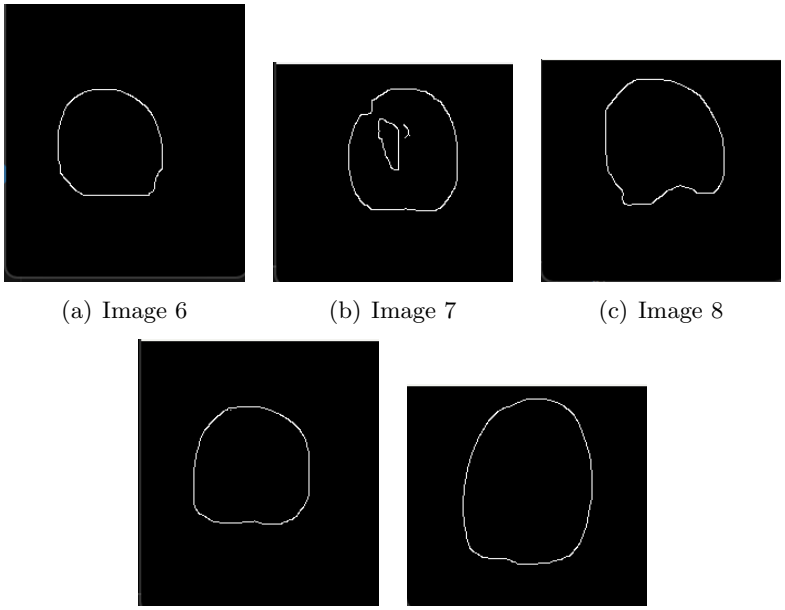
3.4.2 The Canny Edge Detector

The Canny edge detection algorithm is a multi-stage process consisting of:

- Noise reduction using Gaussian smoothing
- Calculation of gradient intensity and direction
- Non-maximum suppression to thin out edges
- Hysteresis thresholding to track strong and weak edges

The algorithm produces a binary edge map highlighting significant boundaries. In our task, Canny is applied to the NIR image after bilateral filtering, which helps in maintaining edge fidelity while removing fine noise that may interfere with the gradients.

The choice of Canny is motivated by its ability to detect both sharp and fine edges while remaining less sensitive to noise compared to simpler methods like Sobel or Laplacian filters.



(a) Image 6 (b) Image 7 (c) Image 8

(d) Image 9 (e) Image 10

Figure 30: Canny edge detection

3.5 Morphological Filtering and Region Refinement

3.5.1 Morphological Closing

The raw edge map produced by Canny is often fragmented, with broken lines and small gaps. To transform these into usable blobs, we apply morphological closing using a cross-shaped kernel. Closing connects nearby edge fragments and ensures that most defects form enclosed boundaries, a requirement for contour analysis.

3.5.2 Background Removal via Flood Fill

To isolate only internal defect regions, we remove the background from the edge map. This is done by performing a flood fill operation starting from the borders of the mask. This process fills the background, and its subtraction

from the fruit mask results in a map of possible regions of internal defects only.

This operation ensures that outer shadows, stalks, or artifacts connected to the fruit boundary are not misclassified as defects.

3.5.3 Final Cleaning with Morphological Opening

To finalize the binary mask, a morphological opening is applied using a small kernel. This step cleans up noise pixels and ensures only significant defect regions remain, preparing the output for clear visualization.

Figure 31: Defects alone on image 7

3.6 Contour Detection and Defect Visualization

The cleaned binary mask is passed to `cv2.findContours`, which retrieves the outlines of each detected defect region. For each contour, the minimum enclosing circle is computed, a method that encloses the shape in a smooth circular boundary.

This circle is drawn on a blank mask and then overlaid on the RGB image using weighted addition. The result is a clear visualization where every detected defect is highlighted with a soft green ring. This aids human interpretation and can also serve as input for quality grading or further analysis.

Figure 32: Defects circled on image 7

Conclusions

This project explored the use of classical computer vision techniques for automated fruit quality inspection through three focused tasks. In Task 1, we began by segmenting apples from the background using NIR imaging and morphological operations, followed by basic defect detection through image filtering and edge enhancement. This established a reliable framework for separating relevant fruit regions from noise. Task 2 addressed a more complex challenge: russet detection. Since russets are subtle and blend with the fruit's natural skin tone, we used LAB color space transformation, unsupervised K-means clustering, and Mahalanobis distance to isolate these regions based on statistical color distribution, resulting in robust and precise detection. Task 3 shifted attention to kiwis, which presented challenges due to their dark and textured surfaces. Here, we applied Otsu thresholding for segmentation and Canny edge detection for highlighting defects, refined through morphological operations. Each task demonstrated the adaptability of traditional image processing methods when tailored carefully to specific fruit types and defect profiles. While deep learning methods remain a potential future direction, this project showcases that well-engineered, interpretable pipelines can still perform effectively in real-world agricultural inspection scenarios.

Bibliography

- R. Gonzales, R. Woods, Digital Image Processing Third Editionâ, Pearson PrenticeHall, 2008.;
- V. S. Nalwa, A Guided Tour of Computer Visionâ, Addison-Wesley Publishing Company, 1993;
- R. Schalkoff, Digital Image Processing And Computer Vision, Wiley, 1989
- Prof. Luigi Di Stefano lecture slides and notes.